LABORATORY OBSERVATION

Course:	Full Stack Web Development
Course Code:	23CM4121
Experiment No.:	2
Student Name:	P. Govindamma
Roll No.:	A24126552104
Date:	29-08-2026

TASK 3 — JAVASCRIPT, DOM MANIPULATION AND STUDENT PROFILE

1. TITLE

Develop a Dynamic Student Profile Application Using JavaScript and DOM Manipulation

2. AIM

To develop a dynamic Student Profile Application using JavaScript that creates a Student object using user-provided values and dynamically displays the student details on a web page using DOM manipulation.

3. OBJECTIVE

- To understand JavaScript classes and objects.
- To create a Student class with appropriate properties.
- To initialize student details using a constructor.
- To accept student information from the user.
- To create a Student object using the entered values.
- To understand JavaScript event handling.
- To use DOM methods to access HTML elements.
- To dynamically create HTML elements using JavaScript.
- To display student details dynamically on the webpage.
- To apply CSS styling to the dynamically generated student profile.
- To use external HTML, CSS and JavaScript files together in a single web application.

4. THEORY

JavaScript is a scripting language used to add dynamic behavior and interactivity to web pages. It can respond to user actions, manipulate HTML elements, validate input, and dynamically update webpage content.

In this application, a JavaScript class named Student is created. The class contains the following properties: Name, Roll Number, Department, and CGPA. A constructor is used to initialize these properties when a Student object is created. The application also uses the DOM (Document Object

Model) to access and modify HTML elements dynamically.

The basic working flow is: User Input → Button Click → JavaScript Event → Student Object → DOM Manipulation → Student Profile Display.

JavaScript Class: A class provides a blueprint for creating objects. The Student class is used to represent student information.

Constructor: The constructor initializes the properties of each Student object.

Event Listener: The addEventListener() method is used to detect when the Display Profile button is clicked.

DOM Manipulation: JavaScript DOM methods such as getElementById(), createElement(), and appendChild() are used to access existing elements and dynamically create new elements.

CSS Styling: CSS is used to style the main container, input fields, button, and dynamically generated student profile.

5. ALGORITHM / PROCEDURE

1. Create an HTML file containing the student input form.
2. Create input fields for Name, Roll Number, Department, and CGPA.
3. Add a button to display the student profile.
4. Create a JavaScript file and define a Student class.
5. Define the constructor with Name, Roll Number, Department, and CGPA parameters.
6. Assign the received values to the Student object properties.
7. Access the display button using getElementById().
8. Access the profile container using getElementById().
9. Add a click event listener to the display button.
10. Read the values entered by the user.
11. Create a new object of the Student class using the entered values.
12. Clear any previously displayed student profile.
13. Dynamically create a heading and paragraph elements for the profile.
14. Add the student details to the created elements.
15. Append the created elements to the profile container.
16. Display the generated student profile on the webpage.
17. Apply CSS styling to the generated profile.
18. Test the application with different sets of input values.
19. Verify the output with the expected student details.

6. PROGRAM

student.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Student Profile</title>
    <link rel="stylesheet" href="student.css">

</head>
<body>
    <div class="container">
        <h1>Student Profile</h1>
        <div class="form">
            <input type="text" id="name" placeholder="Enter Name">
            <input type="text" id="rollNo" placeholder="Enter Roll Number">
            <input type="text" id="department" placeholder="Enter Department">
            <input type="number" id="cgpa" step="0.01" placeholder="Enter CGPA">
            <button id="displayBtn">Display Profile</button>
        </div>
        <div id="profile"></div>
    </div>
    <script src="student.js"></script>
</body>
</html>
```

student.css

```
body {
    font-family: Arial, sans-serif;
    background-color: #f2f2f2;
    margin: 0;
    padding: 50px;
}
.container {
    width: 450px;
    margin: auto;
    padding: 25px;
    background-color: white;
    border-radius: 10px;
    text-align: center;
    box-shadow: 0 4px 10px rgba(0, 0, 0, 0.2);
}
h1 {
    margin-bottom: 20px;
}
input {
    width: 90%;
    padding: 10px;
    margin: 8px 0;
    border: 1px solid #ccc;
    border-radius: 5px;
    font-size: 15px;
}
button {
    margin-top: 15px;
    padding: 10px 20px;
    background-color: #333;
    color: white;
    border: none;
    border-radius: 5px;
    cursor: pointer;
}
button:hover {
    background-color: #555;
}
#profile {
    margin-top: 25px;
    padding: 20px;
    background-color: #f5f5f5;
    border-radius: 8px;
    text-align: left;
}
#profile h2 {
    text-align: center;
```

```
}
#profile p {
    font-size: 16px;
    margin: 10px 0;
}
```

student.js

```
class Student {
    constructor(name, rollNo, department, cgpa) {
        this.name = name;
        this.rollNo = rollNo;
        this.department = department;
        this.cgpa = cgpa;
    }
}
// Selecting the button and profile section
const button = document.getElementById("displayBtn");
const profile = document.getElementById("profile");
// Adding click event
button.addEventListener("click", function () {
    // Getting values from input fields
    let name = document.getElementById("name").value;
    let rollNo = document.getElementById("rollNo").value;
    let department = document.getElementById("department").value;
    let cgpa = document.getElementById("cgpa").value;
    // Creating Student object
    let student = new Student(
        name,
        rollNo,
        department,
        cgpa
    );
    // Creating HTML elements dynamically
    let heading = document.createElement("h2");
    heading.textContent = "Student Profile";
    let nameText = document.createElement("p");
    nameText.textContent = "Name : " + student.name;
    let rollText = document.createElement("p");
    rollText.textContent = "Roll No : " + student.rollNo;
    let departmentText = document.createElement("p");
    departmentText.textContent =
        "Department : " + student.department;
    let cgpaText = document.createElement("p");
    cgpaText.textContent = "CGPA : " + student.cgpa;
    // Clear previous output
    profile.innerHTML = "";
    // Add elements to the webpage
    profile.appendChild(heading);
    profile.appendChild(nameText);
    profile.appendChild(rollText);
    profile.appendChild(departmentText);
    profile.appendChild(cgpaText);
});
```

7. CODE EXPLANATION

Code	Explanation
`class Student`	Defines a class named Student.
`constructor()`	Initializes the properties of a Student object.
`this.name`	Stores the student's name.
`this.rollNo`	Stores the student's roll number.
`this.department`	Stores the student's department.
`this.cgpa`	Stores the student's CGPA.
`getElementById()`	Selects an HTML element using its ID.
`addEventListener()`	Detects and responds to user events such as button clicks.
`.value`	Retrieves the value entered in an input field.
`new Student()`	Creates an object of the Student class.
`createElement()`	Dynamically creates a new HTML element.
`textContent`	Sets the text content of an HTML element.
`innerHTML = ""`	Clears previously generated content before displaying a new profile.
`appendChild()`	Adds a dynamically created element to another element.
`<link rel="stylesheet" href="student.css">`	Links the external CSS file to style the page.
`<script src="student.js"></script>`	Links the external JavaScript file containing the application logic.
`#profile`	The container element where the generated student profile is displayed.
`button:hover`	Applies styling when the mouse pointer is over the button.

8. TEST CASES AND EXECUTION DATA

Valid Input Test Cases

Test Case	Name	Roll No.	Department	CGPA	Expected Result	Actual Result	Status
TC-01	Sneha Reddy	A23126552004	CSE	8.75	Student profile should be displayed	Student profile displayed correctly	Pass
TC-02	Kiran Kumar	A23126552005	ECE	7.90	Student profile should be displayed	Student profile displayed correctly	Pass
TC-03	Meena Devi	A23126552006	EEE	9.25	Student profile should be displayed	Student profile displayed correctly	Pass
TC-04	Rohit Sharma	A23126552007	Mechanical	8.10	Student profile should be displayed	Student profile displayed correctly	Pass
TC-05	Anjali Rao	A23126552008	Civil	7.65	Student profile should be displayed	Student profile displayed correctly	Pass

Note: student.js in this version does not include an empty-field alert check, so a dedicated validation test case is not applicable here; if a field is left blank, the profile is generated with that field shown as blank.

9. OUTPUT

Output 1: Student Information Input Page

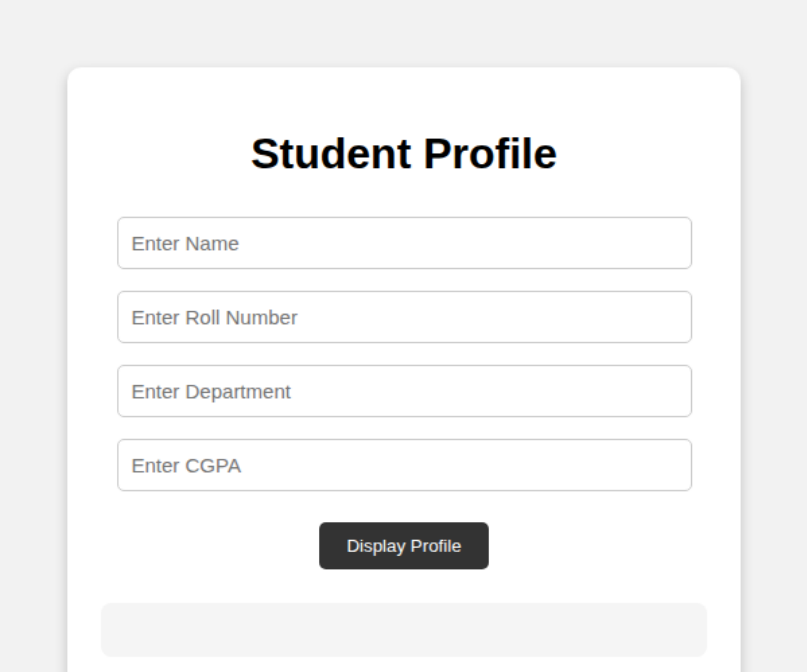

Figure 1: Student Profile application before displaying the student details.

Output 2: Dynamically Generated Student Profile

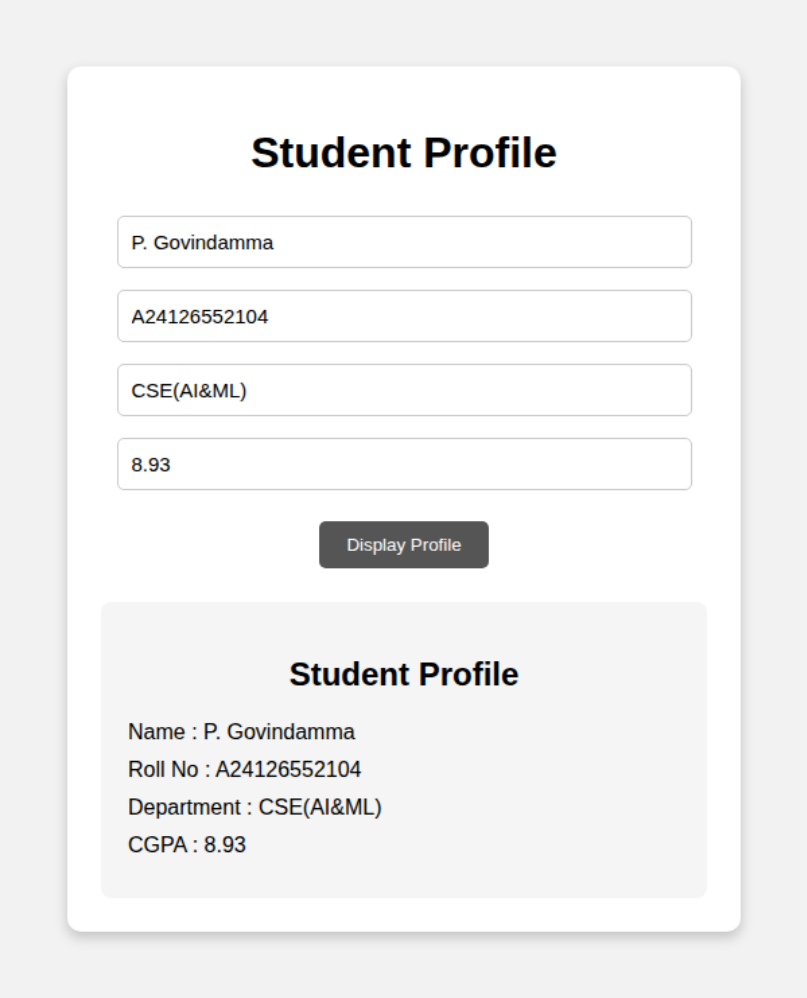

Figure 2: Student Profile dynamically generated using JavaScript DOM manipulation.

10. INTERVIEW QUESTIONS

Q1. What is a JavaScript class?

Answer: A JavaScript class is a blueprint for creating objects. It defines the properties and methods that objects created from the class can have.

Q2. What is the purpose of a constructor in a JavaScript class?

Answer: A constructor is a special method that is automatically called when an object is created. It is used to initialize the properties of the object.

Q3. What is DOM manipulation?

Answer: DOM manipulation is the process of using JavaScript to access, create, modify, or remove HTML elements dynamically in a web page.

Q4. What is the purpose of createElement()?

Answer: The createElement() method creates a new HTML element dynamically using JavaScript. In this application, it is used to create the student profile and its contents.

Q5. What is the purpose of appendChild()?

Answer: The appendChild() method adds a newly created HTML element as a child of an existing element.

Q6. Why is addEventListener() used in this application?

Answer: addEventListener() is used to detect the button click event and execute the code that creates and displays the student profile.

Q7. What is the difference between innerHTML and textContent?

Answer: innerHTML sets or returns the HTML markup contained within an element, so it can parse tags, whereas textContent sets or returns only the plain text, treating any markup as literal characters. textContent is used here to safely insert the student's details as plain text.

11. RESULT

Thus, a Dynamic Student Profile Application was successfully developed using JavaScript classes, objects, event handling, and DOM manipulation. Student details were accepted from the user, a Student object was created dynamically, and the student profile was generated and displayed on the webpage with CSS styling. The application was successfully tested with multiple sets of input values.